



GESTIÓN DE LA
CONFIGURACIÓN DEL
SOFTWARE

Proyecto 2P

Manual de Operaciones: depcheck

Guía de construcción, ejecución y mantenimiento

Integrantes:

Tipan Anton Cesar
Ayovi Villafuerte Camillie
Cordova Erick
Aguilar Mateo (Grupo D)

Carrera:

Software

Materia:

Gestión de la Configuración
del Software

Docente:

Ph.D. Franklin Parrales Bravo

Fecha:

11 de julio de 2026

Indice



1.	Requisitos previos	3
2.	Obtener el proyecto	3
3.	Construccion del proyecto	3
4.	Ejecucion	4
5.	Interpretacion del reporte	4
6.	Mantenimiento de la base de advisories	5
7.	Resolucion de problemas	5

1. Requisitos previos



Para construir y ejecutar depcheck se necesita un entorno con las siguientes herramientas instaladas y disponibles en el PATH.

- Java Development Kit 21 o superior (se verifica con `java -version`).
- Apache Maven 3.9 o superior (se verifica con `mvn -version`).
- Git, solo si se obtiene el proyecto desde el repositorio remoto.
- Conexion a internet en la primera construccion, para que Maven descargue las dependencias al repositorio local.

2. Obtener el proyecto



El código se puede obtener de dos maneras equivalentes.

1. Desde el repositorio: `git clone` del repositorio privado del Grupo D y luego entrar a la carpeta `depcheck`.
2. Desde el paquete de entrega: descomprimir `MAVEN_GrupoD.zip` y entrar a la carpeta `depcheck`.

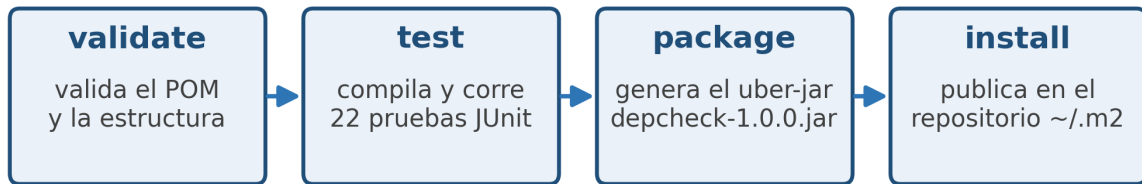
3. Construccion del proyecto



Las fases del ciclo de vida de Maven se ejecutan desde la raíz del proyecto (la carpeta que contiene el `pom.xml`). Cada fase incluye a las anteriores. La Figura 1 resume la secuencia.

1. `mvn validate`: comprueba que el `pom.xml` y la estructura sean correctos.
2. `mvn test`: compila y ejecuta las 22 pruebas unitarias.
3. `mvn package`: genera el artefacto ejecutable `target/depcheck-1.0.0.jar`.
4. `mvn install`: publica el artefacto en el repositorio local `~/.m2`.
5. `mvn clean`: opcional, elimina la carpeta `target` antes de una construcción limpia.

Ciclo de vida Maven ejecutado (acumulativo)



Cada fase incluye a las anteriores: `install` reejecuta validate + test + package.

Orden de las fases de Maven usadas para construir depcheck.

4. Ejecucion

Una vez empaquetado, el programa se ejecuta con el interprete de Java sobre el jar generado. Admite dos modos de uso.

El archivo de entrada contiene una coordenada GAV por linea, con el formato groupId:artifactId:version. Las lineas vacias y las que empiezan con el simbolo numeral se ignoran.

- Sin argumentos: `java -jar target/depcheck-1.0.0.jar` usa la lista de ejemplo embebida.
- Con un archivo propio: `java -jar target/depcheck-1.0.0.jar mis-deps.txt` analiza las dependencias de ese archivo.

5. Interpretacion del reporte

El programa imprime en pantalla cada dependencia analizada y, al final, un reporte con los hallazgos y un resumen. La Figura 2 muestra un ejemplo de salida.

- Cada hallazgo indica la severidad, la coordenada afectada, el identificador CVE y la version en la que se corrige.
- El resumen informa cuantas dependencias se escanearon, cuantos hallazgos hubo por severidad y cuantas dependencias quedaron seguras.
- Codigo de salida 0: no se encontraron vulnerabilidades. Codigo de salida 1: al menos una dependencia es vulnerable. Esto permite usar la herramienta dentro de un pipeline de integracion continua.

```

Reporte final de depcheck

===== REPORTE depcheck =====
[CRITICAL] org.apache.logging.log4j:log4j-core:2.14.1 -> CVE-2021-44228 : actualizar a
    >= 2.15.0
    Log4Shell: ejecucion remota de codigo via lookups JNDI en mensajes de log.
[HIGH] org.apache.logging.log4j:log4j-core:2.14.1 -> CVE-2021-45046 : actualizar a >=
    2.16.0
    Correccion incompleta de Log4Shell: DoS y RCE en ciertas configuraciones.
[CRITICAL] org.apache.commons:commons-text:1.9.0 -> CVE-2022-42889 : actualizar a >=
    1.10.0
    Text4Shell: interpolacion de cadenas permite ejecucion de codigo.
[HIGH] commons-collections:commons-collections:3.2.1 -> CVE-2015-6420 : actualizar a >=
    3.2.2
    Deserializacion insegura de objetos Java (gadget InvokerTransformer).
[HIGH] org.yaml:snakeyaml:1.30 -> CVE-2022-1471 : actualizar a >= 1.33
    Deserializacion insegura via Constructor por defecto (RCE).

-----
Escaneadas 8 | hallazgos 5 (criticas 2, altas 3, medias 0, bajas 0) | seguras 4
=====

```

Salida sobre la lista de ejemplo, con hallazgos por severidad y el resumen final.

6. Mantenimiento de la base de advisories

La base local de vulnerabilidades esta en el archivo `src/main/resources/advisories.txt`. Para agregar un aviso nuevo se anade una linea con seis campos separados por el simbolo de barra vertical.

- Formato: `cveId | groupId | artifactId | fixedVersion | severity | description`.
- El campo severity acepta CRITICA, ALTA, MEDIA o BAJA (tambien sus equivalentes en ingles).
- La regla del modelo es que el aviso afecta a toda version anterior a `fixedVersion`.
- Tras editar el archivo se reconstruye el proyecto con `mvn package` para incluir el cambio en el jar.

7. Resolucion de problemas

- Error de version de Java: verificar que `java -version` reporte 21 o superior.

- Maven no descarga dependencias: revisar la conexión a internet y la configuración de proxy en el archivo settings.xml.
- Coordenada GAV inválida: el programa rechaza líneas que no tengan exactamente tres segmentos separados por dos puntos.
- No aparece ningún hallazgo: confirmar que las versiones del archivo de entrada sean anteriores a las corregidas en la base de advisories.